

기말프로젝트 보고서

과목 : 인공지능기초

담당교수 : 김용우

소속 학과 : 시스템반도체공학과

학번 : 201921310

이름 : 이준형

목차

1. 개요.....	3
2. 가설.....	3-4
1) CNN vs DNN.....	3
2) 활성화 함수	4
3. 가설에 따른 결과분석	5-6
1) CNN vs DNN.....	5
2) 활성화 함수	6
4. 결과.....	6-8
1) 네트워크 설명	6-8
2) 프로젝트 결과	8

1. 개요

이 프로젝트는 Cifar10 데이터 세트의 성능을 높이기 위한 프로젝트이다. Cifar10의 데이터 세트의 구성은 10개의 클래스의 60000개 32x32 컬러 이미지로 구성되어 있으며,

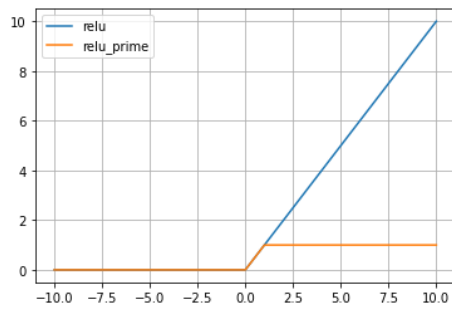
50000개의 훈련 이미지와 10000개의 테스트 이미지로 나뉜다. 50000개의 훈련 이미지만을 활용해 프로그램을 학습시켜 10000개의 테스트 이미지를 더 많이 분별하는 것이 본 목표이다. 여기서 성능을 높이기 위해 프로그램의 신경망 구조를 어떻게 구축하느냐가 문제이다. 성능을 높이기 위한 신경망을 구현하는 방법은 다양하다. 신경망을 구현하기 위한 활성화 함수, 신경망의 층의 개수, 가중치 조절 등 많은 경우의 수가 존재하며, 다음과 같은 가설을 설계하였다.

2. 가설

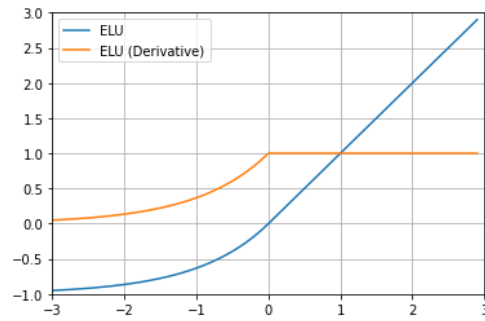
1) CNN vs DNN

CNN 네트워크 구현은 성능을 향상할 것이다. CNN 네트워크는 합성곱 연산을 필터 연산을 처리하는 합성곱 계층과 가로,세로 방향의 공간을 줄이는 풀링 계층을 구성한 네트워크이다. 기본적인 네트워크인 DNN의 네트워크 구성은 신경망의 순저파 때 행렬의 곱을 수행하는 Affine 계층과 정규화하는 Softmax 계층으로 구성되어 있다. DNN과 CNN의 차이점은 이미지 처리 방식에 있다. DNN의 이미지 처리 방식은 2,3차원의 이미지 데이터를 1차원의 한줄 데이터로 변환하여 연산한다. 이미지를 회손시킴으로써 DNN은 2,3차원에서 오는 공간적 정보 손실이 발생하여 성능을 올리는데 방해요소로 작용할 것이라 생각한다. 반면 CNN은 이미지를 그대로 받음으로써 공간적 정보를 유지 할 수 있으며 합성곱 계층을 통해 한 픽셀과 그 주변 픽셀들의 연관성 또한 데이터화 할 수 있다. 즉, CNN 네트워크 구현은 성능을 올리는 데 중요한 역할일 것이라 추측한다.

2) 활성화 함수



[사진1]



[사진2]

Elu 함수는 Leru 함수보다 성능이 올라갈 것이다. 활성화 함수는 입력신호의 총합을 출력 신호로 변환하는 함수를 정의한다. 여기서 중요한 점은 활성화 함수의 종류에 따라 쓰임새가 달라진다는 점이다. Leru 함수는 위 [사진1]을 보면 알듯이 0보다 작으면 0을 크면 출력 값은 x를 가지는 특징이 있다. Leru 함수의 장점은 간단한 개형을 통해 쉽고 빠른 연산이 가능하다. 또한 미분의 표현 또한 단순하여 신경망 계층을 구현하기에 좋은 함수이다. 하지만 Leru 함수의 특징은 단점이 되기도 한다. Dying Leru Leru 함수에서 입력 값이 0보다 작을 경우 기울기가 0이 되어 뉴런이 죽게 된다. 만일 음수의 데이터 값이 신경망 학습의 중요한 데이터라면 Leru 함수를 쓰기에는 다소 조심해야 한다. Leru 함수를 대체할 활성화 함수로 Elu 함수를 고려해 보았다. [사진2]는 Elu 함수의 그래프이다. Elu 함수는 앞서 Leru의 음수 데이터를 보완 하기 위해 나온 함수이다. 음수 데이터를 0으로 계산함을 막음으로써 뉴런이 죽는 것을 방지 할 수 있다. 즉, 활성화 함수로 Elu 함수의 활용은 성능을 올리는 데 중요한 역할일 것이라 추측한다.

3. 가설에 따른 결과분석

1) CNN vs DNN

네트워크 종류		학습횟수	활성함수	train_acc[%]	test_acc[%]
DNN	two_layer_net	10000	Relu	47.16	43.74
		100000	Relu	73.14	48.42
	Three_layer_net	10000	Relu	45.22	44.3
		100000	Relu	66.72	51.85
	Four_layer_net	10000	Relu	19.87	19.76
		100000	Relu	44.89	44.35
		500000	Relu	62.42	51.35
CNN	Three_layer_net	10000	Relu	65.46	58.69
		10000	Elu	76.41	66.13

[표1] DNN과 CNN 결과

위 DNN과 CNN의 test_acc 값을 비교하는 표이다. 이 결과들을 도출하기에 앞서 학습횟수와 은닉층의 개수를 제외하고는 동일한 조건으로 수행하였다. 학습횟수를 동일하게 하지 않는 이유는 은닉층의 개수가 증가함에 따라 요구되는 학습 횟수가 증가하는 것을 확인할 수 있었다. [표1]에서 같은 학습횟수 10000회를 놓고 은닉층을 증가시켰을 때를 비교하면 two_layer_net은 43.74%, Three_layer_net은 44.3%, Four_layer_net은 19.76%의 결과 값을 도출할 수 있다. 이와 같은 결과는 성능을 비교하는데 있어 부정적인 부분이라 생각하여 학습횟수의 변화를 주었다. 위의 결과를 분석하면 DNN만을 기준으로 보면, 은닉층의 증가는 test_acc을 높이는데 도움이 되는 모습을 보인다. 하지만 은닉층의 증가함에 따라 성능의 향상률은 점차 낮아지는 모습을 보인다. 이것으로 DNN을 통해 얻을 수 있는 성능 기댓값은 이 프로젝트를 진행하는데 앞서 좋은 네트워크가 되지 못 한다. 반면 CNN와 DNN의 모든 test_acc 값을 비교해 보았을 때, CNN로 구현된 프로그램의 성능이 현저히 뛰어나다는 것을 확인 할 수 있다. 이것을 통해, 이 프로젝트를 진행함에 있어 DNN 네트워크 구현보다 CNN 네트워크 구현이 상대적으로 좋은 성능을 만들 수 있다는 것을 알 수 있었다.

2) 활성화 함수

네트워크 종류	실행횟수	활성화 함수	train_acc[%]	test_acc[%]
two_layer_net	10000	Relu	47.16	43.73

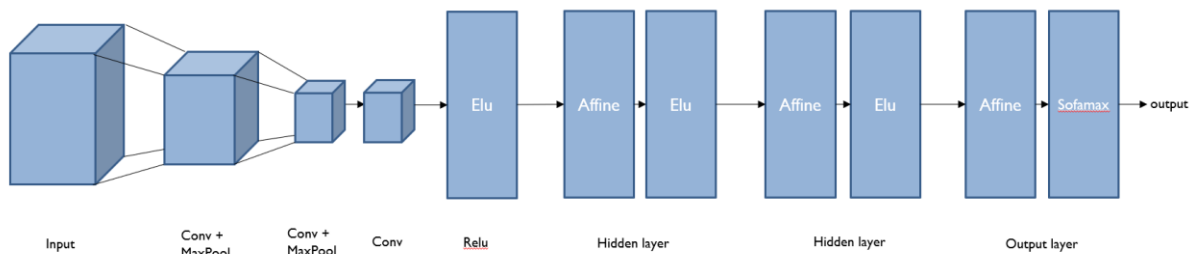
	10000	Elu	49.28	45.41
Four_layer_net	500000	Relu	62.42	51.35
	500000	Elu	68.05	52.17

[표2]

위 [표2]는 DNN에서 네트워크마다 Relu와 Elu의 비교한 표이다. 다른 네트워크로 구현에 따라 결과 값의 차이를 확인하기 위해 two_layer_net과 Four_layer_net으로 준비했다. 각 네트워크마다 실행횟수는 고정으로 하고 활성화 함수를 변화를 주어 측정하였다. 앞서 예측했듯이 두 네트워크 마다 Elu 함수 쪽이 더 좋은 성능을 보여 주었다. 이것으로 이 프로젝트에서 Elu함수는 Relu함수보다 활성화 함수로써 좋은 결과를 보여줄 수 있다는 걸 확인할 수 있었다.

4. 결과

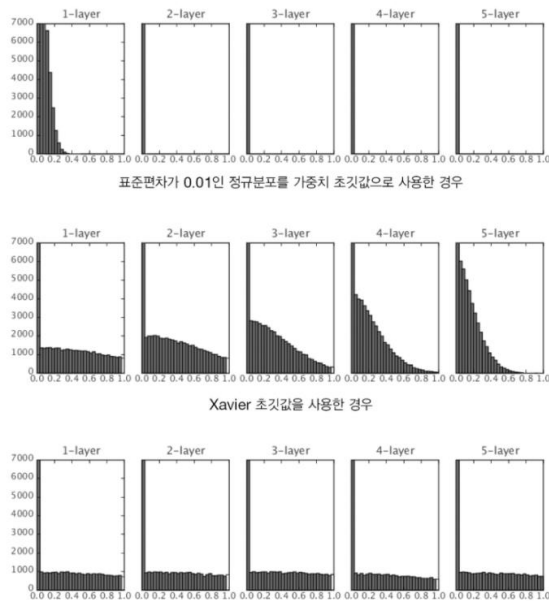
1) 네트워크 설명



[사진3] 네트워크 구성

-CNN 네트워크 : 다음 [사진3]은 이 프로젝트의 네트워크 구성이다. 위 가설과 그에 따른 실험과를 바탕으로 설계했다. CNN 네트워크를 바탕으로 합성곱 계층과 풀링 계층을 활용하였다. 여기서 합성곱 계층과 풀링 계층을 거듭하며 마지막으로 완전연결 계층을 거쳐 결과를 출력한다. 이렇게 설계하게 되면 데이터를 최소화시키면서 효율적으로 시간을 활용하기 위함이다. 또한 완전연결 계층을 보면 three_layer_net을 선택했는데 이것은 위 CNN과 DDN을 비교하는 과정에서 선택하였다. 그저 DDN을 분석하는 과정에

서 two_layer_net, three_layer_net, four_layer_net을 비교하였다. [표1]을 보게되면 물론 은닉층이 증가함에 따라 좋은 성능 효율을 뽑을 수 있다는 것을 알 수 있었지만 비교적 four_layer_net은 three_layer_net보다 월등한 성능을 내지 못 했다. 이것은 같은 성능에서 four_layer_net보다 three_layer_net이 빠른 연산을 하기에 선택였다.



[사진4] 가중치 초기값에 따른 활성화값 분포 변화

가중치 초기값	train_acc[%]	test_acc[%]
0.01	76.41	66.13
$\sqrt{\frac{1}{n}}$ ($n = \text{앞 계층의 노드수}$)	81.11	66.28

[표3] 가중치 초기값에 따른 성능 변화

1	self.params = {}
2	self.params['W1'] = weight_init_std * np.random.randn(6, 3, 5, 5)
3	self.params['b1'] = np.zeros(6)
4	self.params['W3'] = weight_init_std * np.random.randn(16, 6, 5, 5)
5	self.params['b3'] = np.zeros(16)
6	self.params['W5'] = weight_init_std * np.random.randn(120, 16, 5, 5)

7	self.params['b5'] = np.zeros(120)
8	self.params['W6'] = np.sqrt(2/120) * np.random.randn(120, 480)
9	self.params['b6'] = np.zeros(480)
10	self.params['W7'] = np.sqrt(2/480) * np.random.randn(480, 480)
11	self.params['b7'] = np.zeros(480)
12	self.params['W8'] = np.sqrt(2/480) * np.random.randn(480, 480)
13	self.params['b8'] = np.zeros(480)
14	self.params['W9'] = np.sqrt(2/480) * np.random.randn(480, 10)
15	self.params['b9'] = np.zeros(10)

[코드1] 가중치 초기값 설정하는 코드일부

-가중치 초기값 : 가중치는 He 초기값을 적용하였다. 계층을 지나면서 가중치는 주요한 연산이다. He 초기값을 선택한 이유는 기울기 손실 문제이다. 가중치가 0.01인 경우와 Xavier 초기값을 사용하는 경우의 분포도를 보면 분포도는 왼쪽으로 치우쳐지는 모습을 보인다. 이러한 치우침은 층이 깊어지수록 활성화값들의 치우침 또한 커진다. 반면에, He 초기값은 균일한 분포를 만들며 학습에서 일어나는 기울기 손실을 막을 수 있다. 위 [표3]은 완전연결계층을 He 초기값으로 설정했을 때와 안 했을 때의 비교한 표이다. 성능은 0.1%라는 작은 차이를 보이지만 확실히 he 가중치 초기값을 적용했을 때 효과적인 모습을 보인다.

-활성화 함수 : Elu를 활성화 함수로 선택하였다. Elu는 위 [표2]에서 증명했듯이 모든 활성화 함수 계층에 적용하였다. Relu에서 나오는 단점을 제거하며 성능을 올리는데 도움이 된다고 생각해 선택하였다.

2) 결과

```
Parameter Load Complete!
test acc | 66.2200 %
```